

1.SINGLE INHERITANCE

Objective : To write a Java program to demonstrate Single Inheritance.

Steps

1. Start the program.
 2. Create a class A.
 3. Declare an integer variable $x = 10$ in class A.
 4. Define the `add()` method in class A to calculate $x + 20$.
 5. Create class B and extend class A.
 6. Declare an integer variable $y = 200$ in class B.
 7. Define the `sub()` method in class B to calculate $y - 20$.
 8. Create the `main()` method inside the Single class.
 9. Create an object `obj` of class B.
 10. Access the inherited variable x using `obj.x`.
 11. Call the inherited `add()` method using `obj.add()`.
 12. Call the `sub()` method using `obj.sub()`.
 13. Display the results.
 14. Stop the program.
-

2. Algorithm

Algorithm: Single Inheritance

Step 1: Start.

Step 2: Define class A with integer variable $x = 10$.

Step 3: Define the `add()` method to calculate and display $x + 20$.

Step 4: Define class B using `extends A`.

Step 5: Declare integer variable $y = 200$ in class B.

Step 6: Define the `sub()` method to calculate and display $y - 20$.

Step 7: Define the `main()` method in class Single.

Step 8: Create an object of class B.

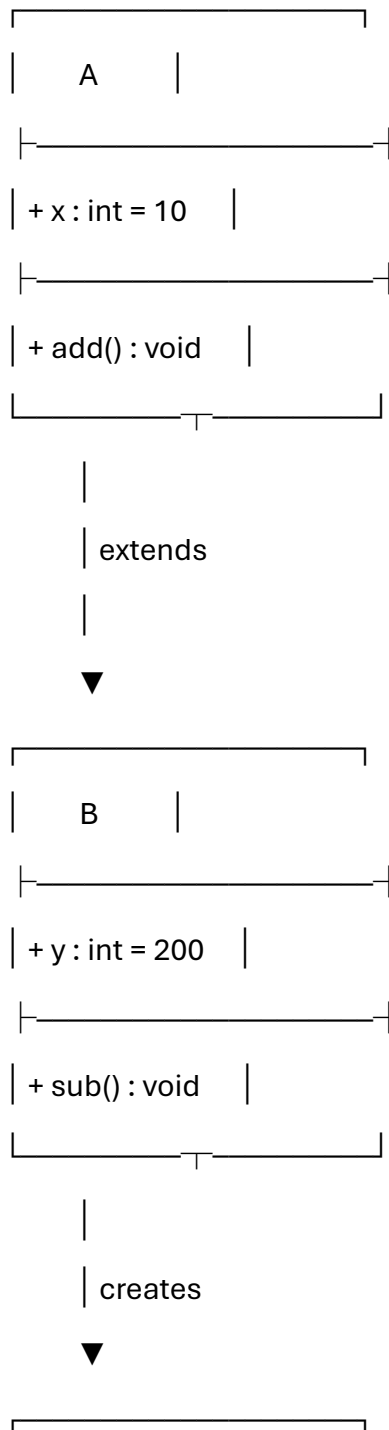
Step 9: Access x and call add() using the object.

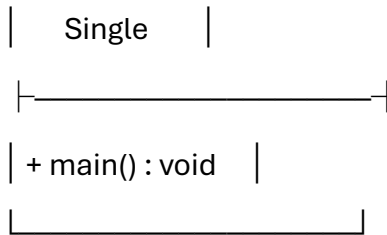
Step 10: Call sub() using the same object.

Step 11: Display the results.

Step 12: Stop.

3. UML Class Diagram





Code :

```

class A{
    int x = 10;

    void add() {
        System.out.println("Add from first is: " + (x+20));
    }
}

class B extends A {
    int y = 200;

    void sub() {
        System.out.println("Sub from second is: " + (y-20));
    }
}

public class Single {
    public static void main(String[] args) {
        B obj = new B();

        System.out.println("Variable : " + obj.x);

        obj.add();

        obj.sub();
    }
}
  
```

Output :

```

PS C:\Users\yabhi\abhi.java>
PS C:\Users\yabhi\abhi.java> c:; cd 'c:\Users\yabhi\abhi.java'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'Single'
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    The method add() is undefined for the type B
    at Single.main(Single.java:17)
PS C:\Users\yabhi\abhi.java>
  
```

Hierarchical Inheritance

Objective : To write a Java program to demonstrate Hierarchical Inheritance.

Steps

1. Start the program.
2. Create a parent class Animal.
3. Define the eat() method in the Animal class.
4. Create a child class Dog that extends Animal.
5. Define the bark() method in the Dog class.
6. Create another child class Cat that extends Animal.
7. Define the meow() method in the Cat class.
8. Create an object d of the Dog class.
9. Call the inherited eat() method using d.
10. Call the bark() method using d.
11. Create an object c of the Cat class.
12. Call the inherited eat() method using c.
13. Call the meow() method using c.
14. Display the results.
15. Stop the program.

2. Algorithm

Algorithm: Hierarchical Inheritance

Step 1: Start.

Step 2: Create a parent class Animal.

Step 3: Define the eat() method in the Animal class.

Step 4: Create class Dog by extending Animal.

Step 5: Define the bark() method in the Dog class.

Step 6: Create class Cat by extending Animal.

Step 7: Define the meow() method in the Cat class.

Step 8: Create an object of the Dog class.

Step 9: Use the Dog object to call eat() and bark().

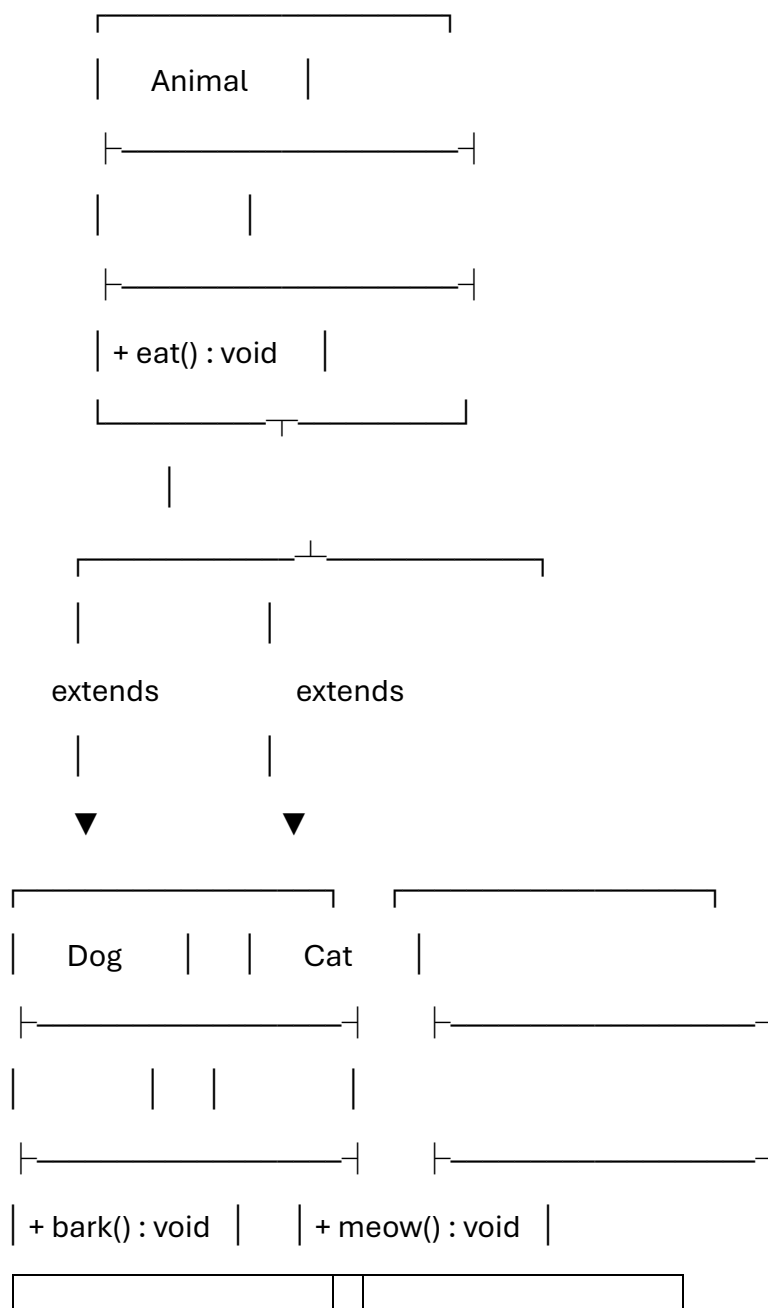
Step 10: Create an object of the Cat class.

Step 11: Use the Cat object to call eat() and meow().

Step 12: Display the output.

Step 13: Stop.

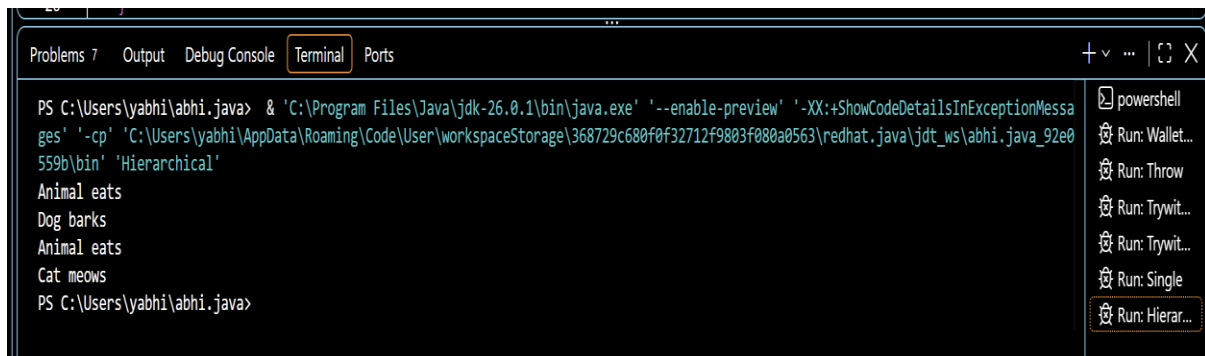
3. UML Class Diagram



Code :

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Hierarchical {  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
  
        Cat c = new Cat();  
        c.eat();  
        c.meow();  
    }  
}
```

Output :



The screenshot shows an IDE's terminal window with the 'Terminal' tab selected. The command prompt is a PowerShell session. The command executed is: `PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'Hierarchical'`. The output of the command is: `Animal eats`, `Dog barks`, `Animal eats`, and `Cat meows`. The prompt then returns to `PS C:\Users\yabhi\abhi.java>`. On the right side of the terminal, there is a vertical toolbar with icons for running the code in different ways: 'Run: Wallet...', 'Run: Throw', 'Run: Trywit...', 'Run: Trywit...', 'Run: Single', and 'Run: Hierar...'. The 'Run: Hierar...' option is highlighted with a yellow box.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'Hierarchical'
Animal eats
Dog barks
Animal eats
Cat meows
PS C:\Users\yabhi\abhi.java>
```

MULTILEVEL INHERITANCE :

Objective :To write a Java program to demonstrate Multilevel Inheritance.

Steps

1. Start the program.
2. Create the parent class C1.
3. Declare the variable $x = 27$ in class C1.
4. Define the show() method in class C1.
5. Create class C2 that extends class C1.
6. Declare the variable $y = 34$ in class C2.
7. Define the see() method using super.x and this.y.
8. Create class C3 that extends class C2.
9. Declare the variable $z = 12$ in class C3.
10. Define the demo() method using super.x, super.y, and this.z.
11. Define the addition() method to calculate $x + y + z$.
12. Create an object T of class C3.
13. Call the demo() method using T.
14. Call the addition() method using T.
15. Display the results.
16. Stop the program.

2. Algorithm

Algorithm: Multilevel Inheritance

Step 1: Start.

Step 2: Create class C1 with variable $x = 27$ and method show().

Step 3: Create class C2 by extending C1.

Step 4: Declare variable $y = 34$ in C2 and define the see() method.

Step 5: Create class C3 by extending C2.

Step 6: Declare variable $z = 12$ in C3.

Step 7: Define the demo() method using inherited variables.

Step 8: Define the addition() method to calculate $x + y + z$.

Step 9: Create an object T of class C3.

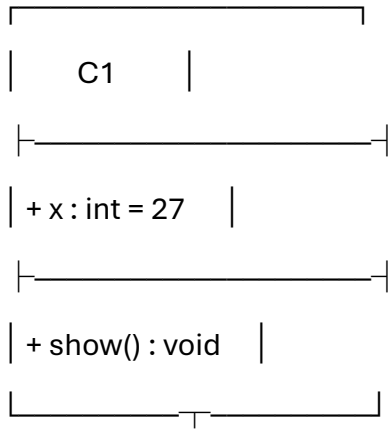
Step 10: Call demo() using object T.

Step 11: Call addition() using object T.

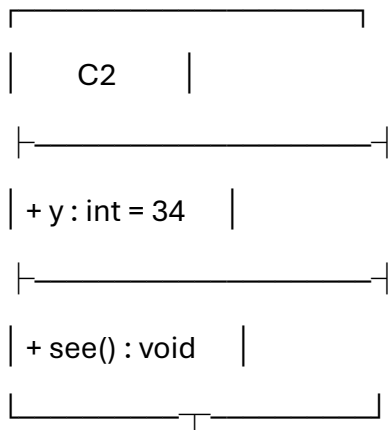
Step 12: Display the output.

Step 13: Stop.

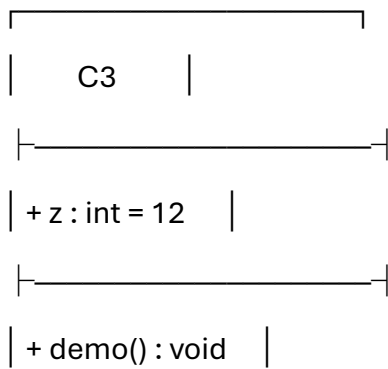
3. UML Class Diagram



extends



extends



```
| + addition() : void |  
|_____|
```

Code :

```
class C1{  
    int x=27;  
    void show() {  
        System.out.println("this is one");  
    }  
}  
  
class C2 extends C1 {  
    int y= 34;  
    void see () {  
        System.out.println("see :"+super.x+ " "+this.y);  
    }  
}  
  
class C3 extends C2{  
    int z=12;  
    void demo() {  
        System.out.println("see :"+super.x+ " "+super.y+ " "+this.z);  
    }  
    void addition (){  
        System.out.println("addition :"+(x+y+z));  
    }  
}  
  
public class Multilevel {  
    public static void main(String[] args) {  
        C3 T=new C3();  
    }  
}
```

```
T.demo();  
T.addition();  
}  
}
```

Output :

see :27 34 12

addition :73

4.HYBRID INHERITANCE :

Objective : To write a Java program to demonstrate Hybrid Inheritance.

Steps

1. Start the program.
2. Create an interface named Animal.
3. Declare the abstract method eat() inside the interface.
4. Create class Dog that implements the Animal interface.
5. Implement the eat() method in the Dog class.
6. Define the bark() method in the Dog class.
7. Create class Puppy that extends the Dog class.
8. Define the play() method in the Puppy class.
9. Create class Cat that implements the Animal interface.
10. Implement the eat() method in the Cat class.
11. Define the meow() method in the Cat class.
12. Create an object p of the Puppy class.
13. Call eat(), bark(), and play() using the Puppy object.
14. Create an object c of the Cat class.
15. Call eat() and meow() using the Cat object.
16. Display the results.
17. Stop the program.

2. Algorithm

Algorithm: Hybrid Inheritance

Step 1: Start.

Step 2: Create an interface Animal with the method eat().

Step 3: Create class Dog that implements Animal.

Step 4: Implement eat() and define bark() in Dog.

Step 5: Create class Puppy that extends Dog.

Step 6: Define the play() method in Puppy.

Step 7: Create class Cat that implements Animal.

Step 8: Implement eat() and define meow() in Cat.

Step 9: Create a Puppy object.

Step 10: Call eat(), bark(), and play() using the Puppy object.

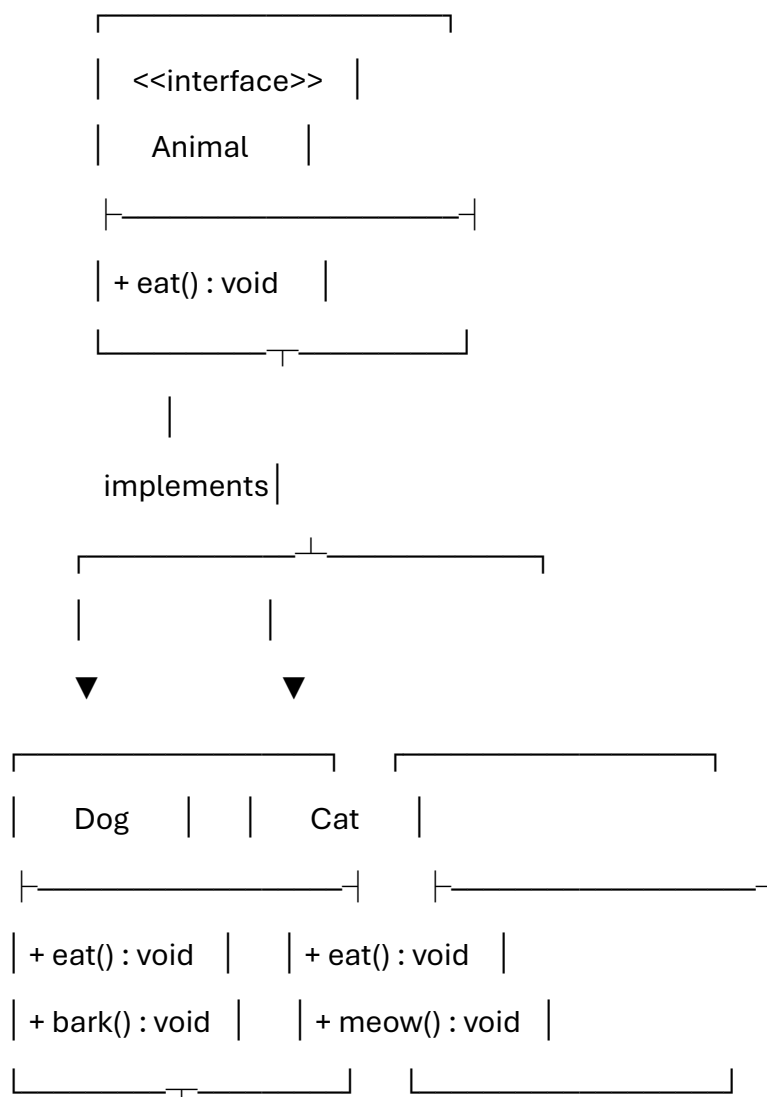
Step 11: Create a Cat object.

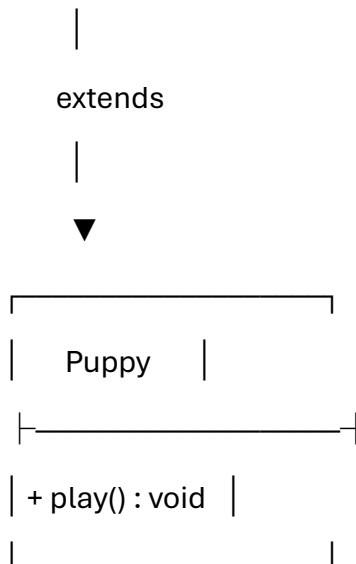
Step 12: Call eat() and meow() using the Cat object.

Step 13: Display the output.

Step 14: Stop.

3. UML Diagram





Code :

```
interface Animal {
    void eat();
}
```

```
class Dog implements Animal {
    public void eat() {
        System.out.println("Dog eats");
    }

    void bark() {
        System.out.println("Dog barks");
    }
}
```

```
class Puppy extends Dog {
    void play() {
        System.out.println("Puppy plays");
    }
}
```

```

class Cat implements Animal {

    public void eat() {

        System.out.println("Cat eats");

    }

    void meow() {

        System.out.println("Cat meows");

    }

}

```

```

public class Hybrid {

    public static void main(String[] args) {

        Puppy p = new Puppy();

        p.eat();

        p.bark();

        p.play();

        Cat c = new Cat();

        c.eat();

        c.meow();

    }

}

```

Output :

```

26 }
27
28 public class Hybrid {
29     public static void main(String[] args) {
30         Puppy p = new Puppy();
31         p.eat();
32         p.bark();
33         p.play();
34         Cat c = new Cat();
35         c.eat();
36         c.meow();
37     }
38 }

```

Problems 7 Output Debug Console Terminal Ports

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'Hybrid'

Animal eats
Dog barks
Puppy plays
Animal eats
Cat meows
PS C:\Users\yabhi\abhi.java>

Run: Throw
Run: Trywit...
Run: Trywit...
Run: Single
Run: Hierar...
Run: Multil...
Run: Hybrid

